8 Securing REST

This chapter covers

- Securing APIs with OAuth 2
- Creating an authorization server
- Adding a resource server to an API
- Consuming OAuth 2–secured APIs

Have you ever taken advantage of valet parking? It's a simple concept: you hand your car keys to a valet near the entrance of a store, hotel, theater, or restaurant, and they deal with the hassle of finding a parking space for you. And then they return your car to you when you ask for it. Maybe it's because I've seen *Ferris Bueller's Day Off* too many times, but I'm always reluctant to hand my car keys to a stranger and hope that they take good care of my vehicle for me.

Nonetheless, valet parking involves granting trust to someone to take care of your car. Many newer cars provide a "valet key," a special key that can be used only to open the car doors and start the engine. This way the amount of trust that you are granting is limited in scope. The valet cannot open the glove compartment or the trunk with the valet key.

In a distributed application, trust is critical between software systems. Even in a simple situation where a client application consumes a backend API, it's important that the client is trusted and anyone else attempting to use that same API is blocked out. And, like the valet, the amount of trust you grant to a client should be limited to only the functions necessary for the client to do its job.

Securing a REST API is different from securing a browser-based web application. In this chapter, we're going to look at OAuth 2, an authorization specification created specifically for API security. In doing so, we'll look at Spring Security's support for OAuth 2. But first, let's set the stage by seeing how OAuth 2 works.

8.1 Introducing OAuth 2

Suppose that we want to create a new back-office application for managing the Taco Cloud application. More specifically, let's say that we want this new application to be able to manage the ingredients available on the main Taco Cloud website.

Before we start writing code for the administrative application, we'll need to add a handful of new endpoints to the Taco Cloud API to support ingredient management. The REST controller in the following listing offers three endpoints for listing, adding, and deleting ingredients.

Listing 8.1 A controller to manage available ingredients

```
package tacos.web.api;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import tacos.Ingredient;
import tacos.data.IngredientRepository;

@RestController
@RequestMapping(path="/api/ingredients", produces="application/json")
@CrossOrigin(origins="http://localhost:8080")
public class IngredientController {

  private IngredientRepository repo;

  @Autowired
  public IngredientController(IngredientRepository repo) {
    this.repo = repo;
  }

  @GetMapping
  public Iterable<Ingredient> allIngredients() {
    return repo.findAll();
  }

  @PostMapping
  @ResponseStatus(HttpStatus.CREATED)
  public Ingredient saveIngredient(@RequestBody Ingredient ingredient) {
    return repo.save(ingredient);
  }
```

```
  @DeleteMapping("/{id}")
  @ResponseStatus(HttpStatus.NO_CONTENT)
  public void deleteIngredient(@PathVariable("id") String ingredientId) {
    repo.deleteById(ingredientId);
  }

}
```

Great! Now all we need to do is get started on the administrative application, calling those endpoints on the main Taco Cloud application as needed to add and delete ingredients.

But wait—there's no security around that API yet. If our backend application can make HTTP requests to add and delete ingredients, so can anyone else. Even using the `curl` command-line client, someone could add a new ingredient like this:

```
$ curl localhost:8080/ingredients \
  -H"Content-type: application/json" \
  -d'{"id":"FISH","name":"Stinky Fish", "type":"PROTEIN"}'
```

They could even use `curl` to delete existing ingredients[1] as follows:

```
$ curl localhost:8080/ingredients/GRBF -X DELETE
```

This API is part of the main application and available to the world; in fact, the `GET` endpoint is used by the user interface of the main application in home.html. Therefore, it's clear that we'll need to secure at least the `POST` and `DELETE` endpoints.

One option is to use HTTP Basic authentication to secure the /ingredients endpoints. This could be done by adding `@PreAuthorize` to the handler methods like this:

```
@PostMapping
@PreAuthorize("#{hasRole('ADMIN')}")
public Ingredient saveIngredient(@RequestBody Ingredient ingredient) {
  return repo.save(ingredient);
}

@DeleteMapping("/{id}")
@PreAuthorize("#{hasRole('ADMIN')}")
public void deleteIngredient(@PathVariable("id") String ingredientId) {
  repo.deleteById(ingredientId);
}
```

Or, the endpoints could be secured in the security configuration like this:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
  http
    .authorizeRequests()
      .antMatchers(HttpMethod.POST, "/ingredients").hasRole("ADMIN")
      .antMatchers(HttpMethod.DELETE, "/ingredients/**").hasRole("ADMIN")

      ...
}
```

Whether or not to use the “ROLE_” prefix

Authorities in Spring Security can take several forms, including roles, permissions, and (as we’ll see later) OAuth2 scopes. Roles, specifically, are a specialized form of authority that are prefixed with `"ROLE_"`.

When working with methods or SpEL expressions that deal directly with roles, such as `hasRole()`, the `"ROLE_"` prefix is inferred. Thus, a call to `hasRole("ADMIN")` is internally checking for an authority whose name is `"ROLE_ADMIN"`. You do not need to explicitly use the `"ROLE_"` prefix when calling these methods and functions (and, in fact, doing so will result in a double `"ROLE_"` prefix).

Other Spring Security methods and functions that deal with authority more generically can also be used to check for roles. But in those cases, you must explicitly add the `"ROLE_"` prefix. For example, if you chose to use `hasAuthority()` instead of `hasRole()`, you’d need to pass in `"ROLE_ADMIN"` instead of `"ADMIN"`.

Either way, the ability to submit `POST` or `DELETE` requests to /ingredients will require that the submitter also provide credentials that have `"ROLE_ADMIN"` authority. For example, using `curl`, the credentials can be specified with the `-u` parameter, as shown here:

```
$ curl localhost:8080/ingredients \
  -H"Content-type: application/json" \
  -d'{"id":"FISH","name":"Stinky Fish", "type":"PROTEIN"}' \
  -u admin:l3tm31n
```

Although HTTP Basic will lock down the API, it is rather . . . um . . . basic. It requires that the client and the API share common knowledge of the user credentials, possibly duplicating information. Moreover, although HTTP Basic credentials are Base64-encoded in the header of the request, if a hacker were to somehow intercept the request, the credentials could easily

be obtained, decoded, and used for evil purposes. If that were to happen, the password would need to be changed, thus requiring an update and reauthentication in all clients.

What if instead of requiring that the admin user identify themselves on every request, the API just asks for some token that proves that they are authorized to access the resources? This would be roughly like a ticket to a sporting event. To enter the game, the person at the turnstiles doesn't need to know who you are; they just need to know that you have a valid ticket. If so, then you are allowed access.

That's roughly how OAuth 2 authorization works. Clients request an access token—analogous to a valet key—from an authorization server, with the express permission of a user. That token allows them to interact with an API on behalf of the user who authorized the client. At any point, the token could expire or be revoked, without requiring that the user's password be changed. In such cases, the client would just need to request a new access token to be able to continue acting on the user's behalf. This flow is illustrated in figure 8.1.

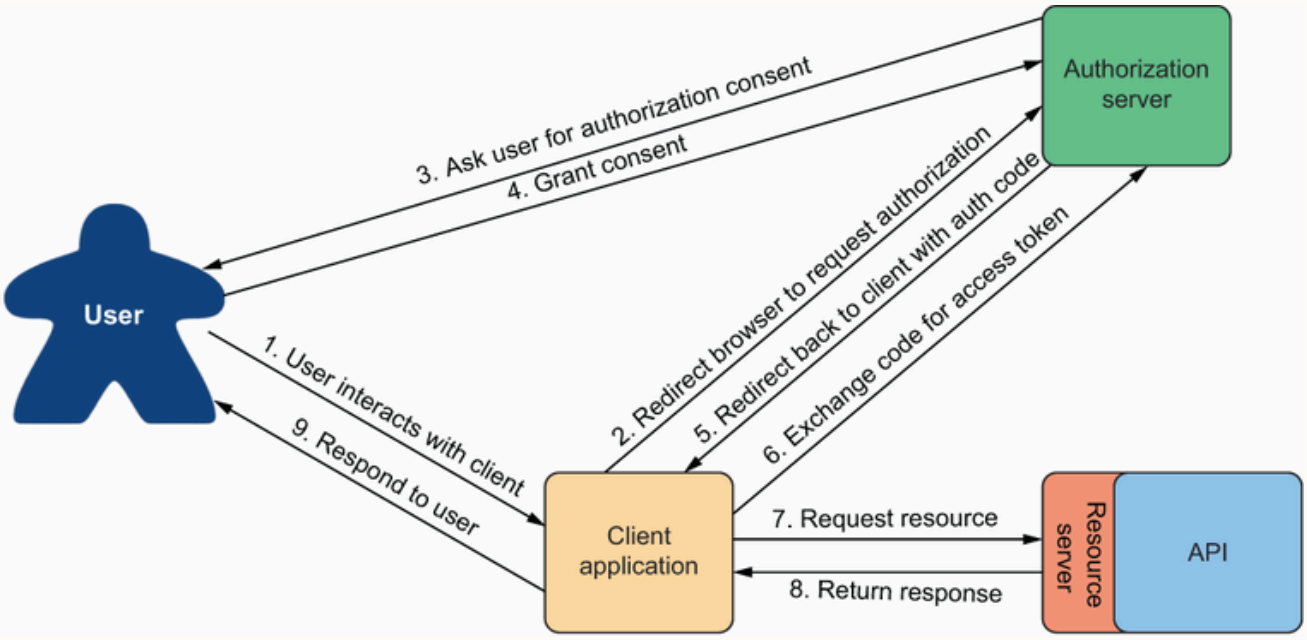


Figure 8.1 The OAuth 2 authorization code flow

OAuth 2 is a very rich security specification that offers a lot of ways to use it. The flow described in figure 8.1 is called *authorization code grant*. Other flows supported by the OAuth 2 specification include these:

- *Implicit grant*—Like authorization code grant, implicit grant redirects the user's browser to the authorization server to get user consent. But when redirecting back, rather than provide an authorization code in the request, the access token is granted implicitly in the request. Although originally designed for JavaScript clients running in a browser, this flow is not generally recommended anymore, and authorization code grant is preferred.

- *User credentials (or password) grant*—In this flow, no redirect takes place, and there may not even be a web browser involved. Instead, the client application obtains the user's credentials and exchanges them directly for an access token. This flow seems suitable for clients that are not browser based, but modern applications often favor asking the user to go to a website in their browser and perform authorization code grant to avoid having to handle the user's credentials.
- *Client credentials grant*—This flow is like user credentials grant, except that instead of exchanging a user's credentials for an access token, the client exchanges its own credentials for an access token. However, the token granted is limited in scope to performing non-user-focused operations and can't be used to act on behalf of a user.

For our purposes, we're going to focus on the authorization code grant flow to obtain a JSON Web Token (JWT) access token. This will involve creating a handful of applications that work together, including the following:

- *The authorization server*—An authorization server's job is to obtain permission from a user on behalf of a client application. If the user grants permission, then the authorization server gives an access token to the client application that it can use to gain authenticated access to an API.
- *The resource server*—A resource server is just another name for an API that is secured by OAuth 2. Although the resource server is part of the API itself, for the sake of discussion, the two are often treated as two distinct concepts. The resource server restricts access to its resources unless the request provides a valid access token with the necessary permission scope. For our purposes, the Taco Cloud API we started in chapter 7 will serve as our resource server, once we add a bit of security configuration to it.
- *The client application*—The client application is an application that wants to consume an API but needs permission to do so. We'll build a simple administrative application for Taco Cloud to be able to add new ingredients.
- *The user*—This is the human who uses the client application and grants the application permission to access the resource server's API on their behalf.

In the authorization code grant flow, a series of browser redirects between the client application and the authorization server occurs as the client obtains an access token. It starts with the client redirecting the user's browser to the authorization server, asking for specific permissions (or "scope"). The authorization server then asks the user to log in and consent to the requested permissions. After the user has granted consent, the authorization server redirects the browser back to the client with a code that the client can then exchange for an access token. Once the client has the access token, it can then be used to interact with the resource server API by passing it in the `"Authorization"` header of every request.

Although we're going to restrict our focus on a specific use of OAuth 2, you are encouraged to dig deeper into the subject by reading the OAuth 2 specification (**https://oauth.net/2/**) or

reading any one of the following books on the subject:

- *OAuth 2 in Action*: **https://www.manning.com/books/oauth-2-in-action**
- *Microservices Security in Action*: **https://www.manning.com/books/microservices-security-in-action**
- *API Security in Action*: **https://www.manning.com/books/api-security-in-action**

You might also want to have a look at a liveProject called "Protecting User Data with Spring Security and OAuth2" (**http://mng.bz/4KdD**).

For several years, a project called Spring Security for OAuth provided support for both OAuth 1.0a and OAuth 2. It was separate from Spring Security but developed by the same team. In recent years, however, the Spring Security team has absorbed the client and resource server components into Spring Security itself.

As for the authorization server, it was decided that it not be included in Spring Security. Instead, developers are encouraged to use authorization servers from various vendors such as Okta, Google, and others. But, due to demand from the developer community, the Spring Security team started a Spring Authorization Server project.[2] This project is labeled as "experimental" and is intended to eventually be community driven, but it serves as a great way to get started with OAuth 2 without signing up for one of those other authorization server implementations.

Throughout the rest of this chapter, we're going to see how to use OAuth 2 using Spring Security. Along the way, we'll create two new projects, an authorization server project and a client project, and we'll modify our existing Taco Cloud project such that its API acts as a resource server. We'll start by creating an authorization server using the Spring Authorization Server project.

8.2 Creating an authorization server

An authorization server's job is primarily to issue an access token on behalf of a user. As mentioned earlier, we have several authorization server implementations to choose from, but we're going to use Spring Authorization Server for our project. Spring Authorization Server is experimental and doesn't implement all of the OAuth 2 grant types, but it does implement the authorization code grant and client credentials grant.

The authorization server is a distinct application from any application that provides the API and is also distinct from the client. Therefore, to get started with Spring Authorization Server, you'll want to create a new Spring Boot project, choosing (at least) the web and security starters. For our authorization server, users will be stored in a relational database using JPA, so be sure to add the JPA starter and H2 dependencies as well. And, if you're using

Lombok to handle getters, setters, constructors, and whatnot, then be sure to include it as well.

Spring Authorization Server isn't (yet) available as a dependency from the Initializr. So once your project has been created, you'll need to manually add the Spring Authorization Server dependency to your build. For example, here's the Maven dependency you'll need to include in your pom.xml file:

```
<dependency>
  <groupId>org.springframework.security.experimental</groupId>
  <artifactId>spring-security-oauth2-authorization-server</artifactId>
  <version>0.1.2</version>
</dependency>
```

Next, because we'll be running this all on our development machines (at least for now), you'll want to make sure that there's not a port conflict between the main Taco Cloud application and the authorization server. Adding the following entry to the project's application.yml file will make the authorization server available on port 9000:

```
server:
  port: 9000
```

Now let's dive into the essential security configuration that will be used by the authorization server. The next code listing shows a very simple Spring Security configuration class that enables form-based login and requires that all requests be authenticated.

Listing 8.2 Essential security configuration for form-based login

```
package tacos.authorization;
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.
        HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.
        EnableWebSecurity;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

import tacos.authorization.users.UserRepository;

@EnableWebSecurity
public class SecurityConfig {
```

```
  @Bean
  SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http)
        throws Exception {
    return http
      .authorizeRequests(authorizeRequests ->
        authorizeRequests.anyRequest().authenticated()
      )

      .formLogin()

      .and().build();
  }

  @Bean
  UserDetailsService userDetailsService(UserRepository userRepo) {
   return username -> userRepo.findByUsername(username);
  }

  @Bean
  public PasswordEncoder passwordEncoder() {
   return new BCryptPasswordEncoder();
  }
}
```

Notice that the `UserDetailsService` bean works with a `TacoUserRepository` to look up users by their username. To get on with configuring the authorization server itself, we'll skip over the specifics of `TacoUserRepository`, but suffice it to say that it looks a lot like some of the Spring Data–based repositories we've created since chapter 3.

The only thing worth noting about the `TacoUserRepository` is that (for convenience in testing) you could use it in a `CommandLineRunner` bean to prepopulate the database with a couple of test users as follows:

```
@Bean
public ApplicationRunner dataLoader(
      UserRepository repo, PasswordEncoder encoder) {
 return args -> {
  repo.save(
    new User("habuma", encoder.encode("password"), "ROLE_ADMIN"));
  repo.save(
    new User("tacochef", encoder.encode("password"), "ROLE_ADMIN"));
 };
}
```

Now we can start applying configuration to enable an authorization server. The first step in configuring an authorization server is to create a new configuration class that imports some common configuration for an authorization server. The following code for `AuthorizationServerConfig` is a good start:

```
@Configuration(proxyBeanMethods = false)
public class AuthorizationServerConfig {

 @Bean
 @Order(Ordered.HIGHEST_PRECEDENCE)
 public SecurityFilterChain authorizationServerSecurityFilterChain(HttpSecurity http) throws Exception {
  OAuth2AuthorizationServerConfiguration
     .applyDefaultSecurity(http);
  return http
     .formLogin(Customizer.withDefaults())
     .build();
 }

  ...

}
```

The `authorizationServerSecurityFilterChain()` bean method defines a `SecurityFilterChain` that sets up some default behavior for the OAuth 2 authorization server and a default form login page. The `@Order` annotation is given `Ordered.HIGHEST_ PRECEDENCE` to ensure that if for some reason there are other beans of this type declared, this one takes precedence over the others.

For the most part, this is a boilerplate configuration. If you want, feel free to dive in a little deeper and customize the configuration. For now, we're just going to go with the defaults.

One component that isn't boilerplate, and thus not provided by `OAuth2AuthorizationServerConfiguration`, is the client repository. A client repository is analogous to a user details service or user repository, except that instead of maintaining details about users, it maintains details about clients that might be asking for authorization on behalf of users. It is defined by the `RegisteredClientRepository` interface (provided by Spring Security), which looks like this:

```
interface RegisteredClientRepository {

   @Nullable
   RegisteredClient findById(String id);

   @Nullable
```

```
    RegisteredClient findByClientId(String clientId);

}
```

In a production setting, you might write a custom implementation of `RegisteredClientRepository` to retrieve client details from a database or from some other source. But out of the box, Spring Authorization Server offers an in-memory implementation that is perfect for demonstration and testing purposes. You're encouraged to implement `RegisteredClientRepository` however you see fit. But for our purposes, we'll use the in-memory implementation to register a single client with the authorization server. Add the following bean method to `AuthorizationServerConfig`:

```
@Bean
public RegisteredClientRepository registeredClientRepository(
      PasswordEncoder passwordEncoder) {
  RegisteredClient registeredClient =
    RegisteredClient.withId(UUID.randomUUID().toString())
      .clientId("taco-admin-client")
      .clientSecret(passwordEncoder.encode("secret"))
      .clientAuthenticationMethod(
          ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
      .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
      .authorizationGrantType(AuthorizationGrantType.REFRESH_TOKEN)
      .redirectUri(
        "http://127.0.0.1:9090/login/oauth2/code/taco-admin-client")
      .scope("writeIngredients")
      .scope("deleteIngredients")
      .scope(OidcScopes.OPENID)
      .clientSettings(
        clientSettings -> clientSettings.requireUserConsent(true))
      .build();
  return new InMemoryRegisteredClientRepository(registeredClient);
}
```

As you can see, there are a lot of details that go into a `RegisteredClient`. But going from top to bottom, here's how our client is defined:

- *ID*—A random, unique identifier.
- *Client ID*—Analogous to a username, but instead of a user, it is a client. In this case, `"taco-admin-client"`.
- *Client secret*—Analogous to a password for the client. Here we're using the word `"secret"` for the client secret.

- *Authorization grant type*—The OAuth 2 grant types that this client will support. In this case, we're enabling authorization code and refresh token grants.
- *Redirect URL*—One or more registered URLs that the authorization server can redirect to after authorization has been granted. This adds another level of security, preventing some arbitrary application from receiving an authorization code that it could exchange for a token.
- *Scope*—One or more OAuth 2 scopes that this client is allowed to ask for. Here we are setting three scopes: `"writeIngredients"`, `"deleteIngredients"`, and the constant `OidcScopes.OPENID`, which resolves to `"openid"`. The `"openid"` scope will be necessary later when we use the authorization server as a single-sign-on solution for the Taco Cloud admin application.
- *Client settings*—This is a lambda that allows us to customize the client settings. In this case, we're requiring explicit user consent before granting the requested scope. Without this, the scope would be implicitly granted after the user logs in.

Finally, because our authorization server will be producing JWT tokens, the tokens will need to include a signature created using a JSON Web Key (JWK)[3] as the signing key. Therefore, we'll need a few beans to produce a JWK. Add the following bean method (and private helper methods) to `AuthorizationServerConfig` to handle that for us:

```
@Bean
 public JWKSource<SecurityContext> jwkSource()
        throws NoSuchAlgorithmException {
  RSAKey rsaKey = generateRsa();
  JWKSet jwkSet = new JWKSet(rsaKey);
  return (jwkSelector, securityContext) -> jwkSelector.select(jwkSet);
 }

 private static RSAKey generateRsa() throws NoSuchAlgorithmException {
  KeyPair keyPair = generateRsaKey();
  RSAPublicKey publicKey = (RSAPublicKey) keyPair.getPublic();
  RSAPrivateKey privateKey = (RSAPrivateKey) keyPair.getPrivate();
  return new RSAKey.Builder(publicKey)
      .privateKey(privateKey)
      .keyID(UUID.randomUUID().toString())
      .build();
 }

 private static KeyPair generateRsaKey() throws NoSuchAlgorithmException {
    KeyPairGenerator keyPairGenerator = KeyPairGenerator.getInstance("RSA");
    keyPairGenerator.initialize(2048);
    return keyPairGenerator.generateKeyPair();
 }

 @Bean
```

```
public JwtDecoder jwtDecoder(JWKSource<SecurityContext> jwkSource) {
  return OAuth2AuthorizationServerConfiguration.jwtDecoder(jwkSource);
}
```

There appears to be a lot going on here. But to summarize, the `JWKSource` creates RSA 2048-bit key pairs that will be used to sign the token. The token will be signed using the private key. The resource server can then verify that the token received in a request is valid by obtaining the public key from the authorization server. We'll talk more about that when we create the resource server.

All of the pieces of our authorization server are now in place. All that's left to do is start it up and try it out. Build and run the application, and you should have an authorization server listening on port 9000.

Because we don't have a client yet, you can pretend to be a client using your web browser and the `curl` command-line tool. Start by pointing your web browser at http://localhost:9000/oauth2/authorize?response_type=code&client_id=tacoadmin-client&redirect_uri= http://127.0.0.1:9090/login/oauth2/code/taco-admin-client&-scope=writeIngredients+deleteIngredients.[4] You should see a login page that looks like figure 8.2.

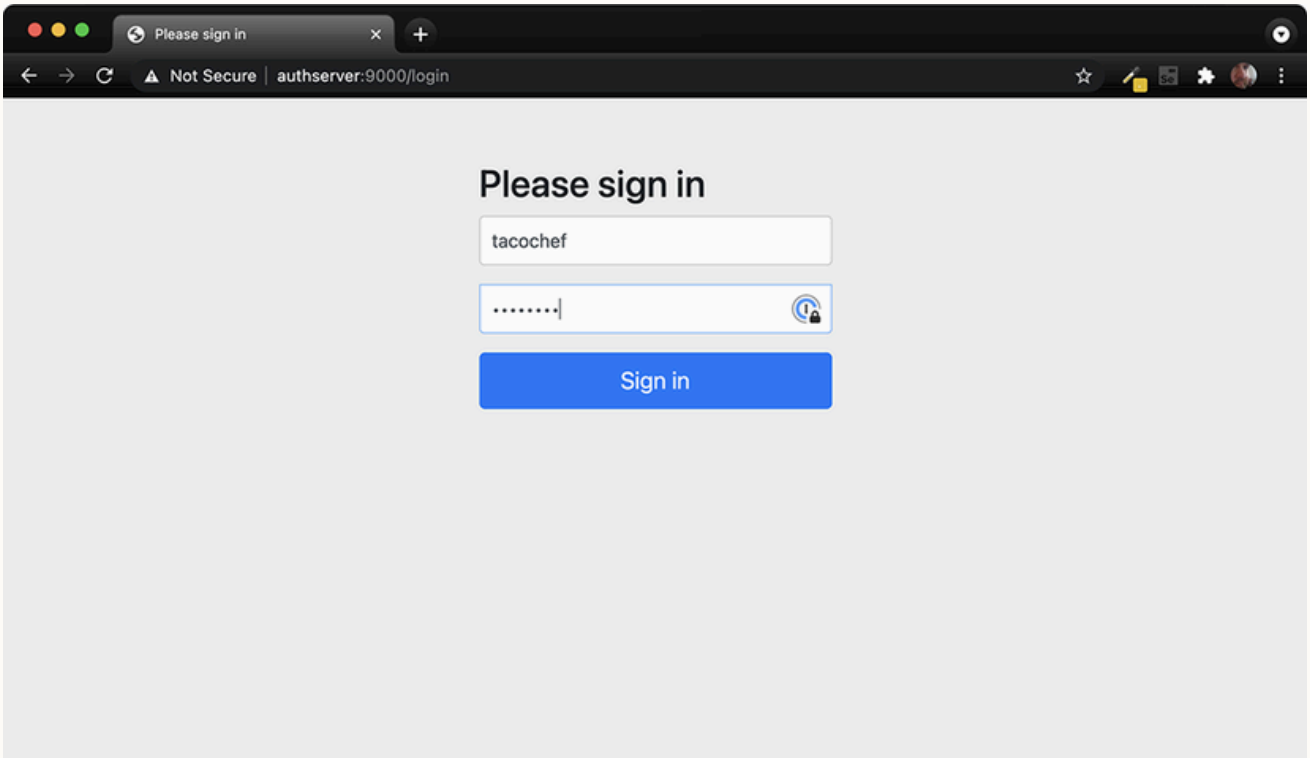


Figure 8.2 The authorization server login page

After logging in (with "tacochef" and "password," or some username-password combination in the database under `TacoUserRepository`), you'll be asked to consent to the requested scopes on a page that looks like figure 8.3.

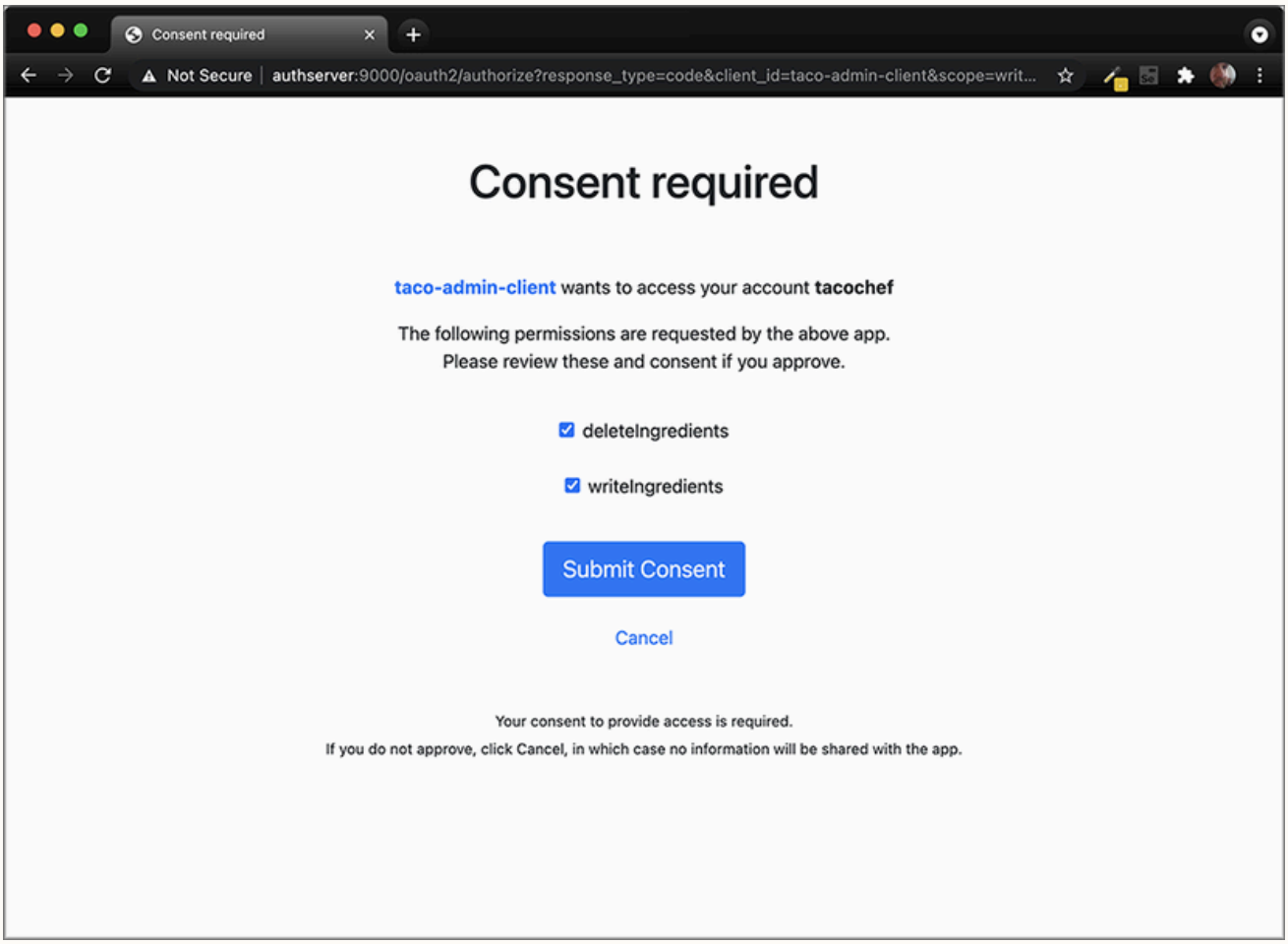


Figure 8.3 The authorization server consent page

After granting consent, the browser will be redirected back to the client URL. We don't have a client yet, so there's probably nothing there and you'll receive an error. But that's OK—we're pretending to be the client, so we'll obtain the authorization code from the URL ourselves.

Look in the browser's address bar, and you'll see that the URL has a `code` parameter. Copy the entire value of that parameter, and use it in the following `curl` command line in place of `$code`:

```
$ curl localhost:9000/oauth2/token \
    -H"Content-type: application/x-www-form-urlencoded" \
    -d"grant_type=authorization_code" \
    -d"redirect_uri=http://127.0.0.1:9090/login/oauth2/code/taco-admin-client" \
    -d"code=$code" \
    -u taco-admin-client:secret
```

Here we're exchanging the authorization code we received for an access token. The payload body is in "application/x-www-form-urlencoded" format and sends the grant type (`"authorization_code"`), the redirect URI (for additional security), and the authorization code itself. If all goes well, then you'll receive a JSON response that (when formatted) looks like this:

```
{
  "access_token":"eyJraWQ...",
  "refresh_token":"HOzHA5s...",
  "scope":"deleteIngredients writeIngredients",
  "token_type":"Bearer",
  "expires_in":"299"
}
```

The `"access_token"` property contains the access token that a client can use to make requests to the API. In reality, it is much longer than shown here. Likewise, the `"refresh_token"` has been abbreviated here to save space. But the access token can now be sent on requests to the resource server to gain access to resources requiring either the `"writeIngredients"` or `"deleteIngredients"` scope. The access token will expire in 299 seconds (or just less than 5 minutes), so we'll have to move quickly if we're going to use it. But if it expires, then we can use the refresh token to obtain a new access token without going through the authorization flow all over again.

So, how can we use the access token? Presumably, we'll send it in a request to the Taco Cloud API as part of the `"Authorization"` header—perhaps something like this:

```
$ curl localhost:8080/ingredients \
  -H"Content-type: application/json" \
  -H"Authorization: Bearer eyJraWQ..." \
  -d'{"id":"FISH","name":"Stinky Fish", "type":"PROTEIN"}'
```

At this point, the token achieves nothing for us. That's because our Taco Cloud API hasn't been enabled to be a resource server yet. But in lieu of an actual resource server and client API, we can still inspect the access token by copying it and pasting into the form at **https://jwt.io**. The result will look something like figure 8.4.

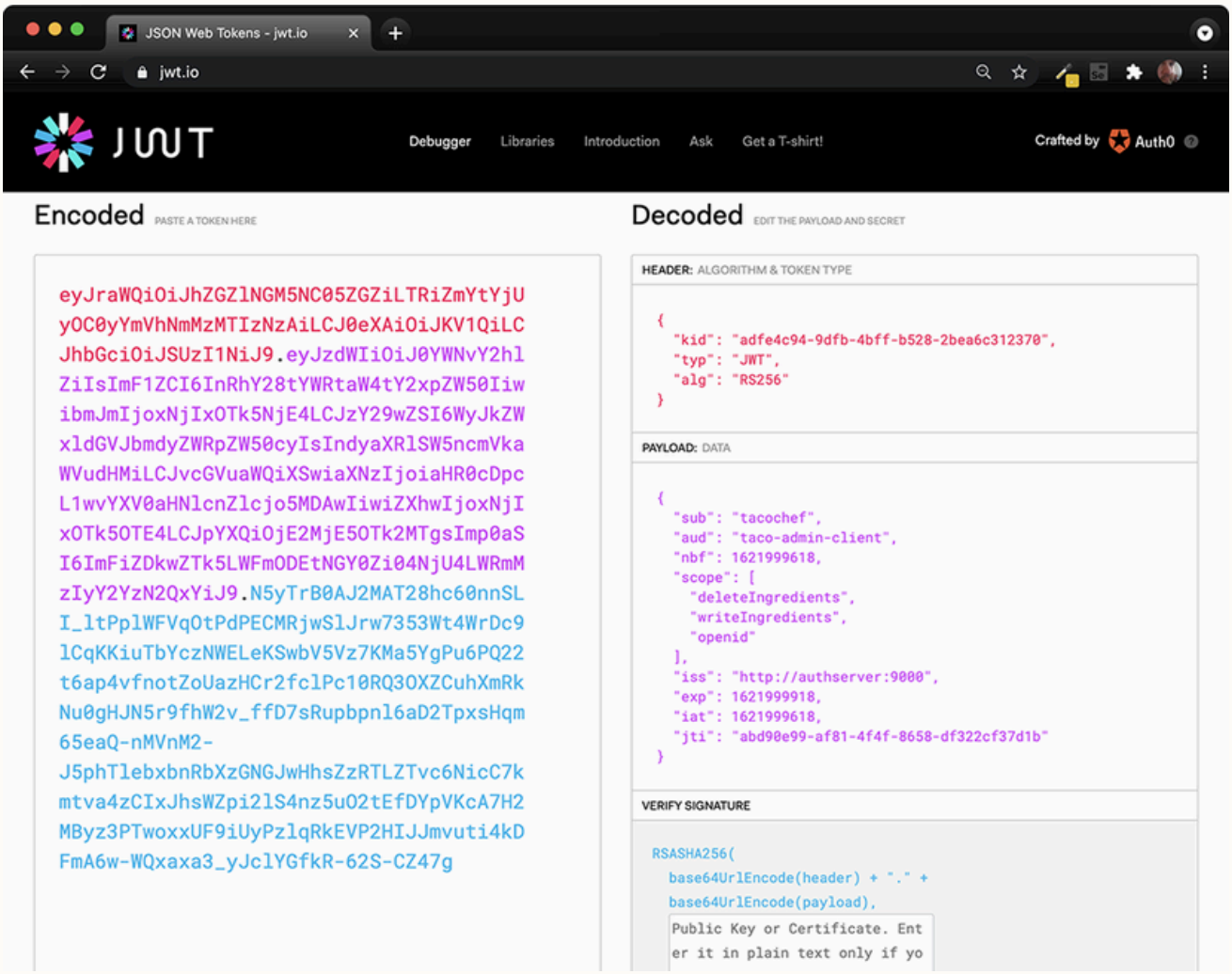


Figure 8.4 Decoding a JWT token at jwt.io

As you can see, the token is decoded into three parts: the header, the payload, and the signature. A closer look at the payload shows that this token was issued on behalf of the user named tacochef and the token has the `"writeIngredients"` and `"deleteIngredients"` scopes. Just what we asked for!

After about 5 minutes, the access token will expire. You can still inspect it in the debugger at **https://jwt.io**, but if it were given in a real request to an API, it would be rejected. But you can request a new access token without going through the authorization code grant flow again. All you need to do is make a new request to the authorization server using the `"refresh_token"` grant and passing the refresh token as the value of the `"refresh_token"` parameter. Using `curl`, such a request will look like this:

```
$ curl localhost:9000/oauth2/token \
    -H"Content-type: application/x-www-form-urlencoded" \
    -d"grant_type=refresh_token&refresh_token=HOzHA5s..." \
    -u taco-admin-client:secret
```

The response to this request will be the same as the response from the request that exchanged the authorization code for an access token initially, only with a fresh new access

token.

Although it's fun to paste access tokens into **https://jwt.io**, the real power and purpose of the access token is to gain access to an API. So let's see how to enable a resource server on the Taco Cloud API.

8.3 Securing an API with a resource server

The resource server is actually just a filter that sits in front of an API, ensuring that requests for resources that require authorization include a valid access token with the required scope. Spring Security provides an OAuth2 resource server implementation that you can add to an existing API by adding the following dependency to the build for the project build as follows:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

You can also add the resource server dependency by selecting the "OAuth2 Resource Server" dependency from the Initializr when creating a project.

With the dependency in place, the next step is to declare that `POST` requests to /ingredients require the `"writeIngredients"` scope and that `DELETE` requests to /ingredients require the `"deleteIngredients"` scope. The following excerpt from the project's `SecurityConfig` class shows how to do that:

```
@Override
 protected void configure(HttpSecurity http) throws Exception {
  http
    .authorizeRequests()
    ...
    .antMatchers(HttpMethod.POST, "/api/ingredients")
      .hasAuthority("SCOPE_writeIngredients")
    .antMatchers(HttpMethod.DELETE, "/api/ /ingredients")
      .hasAuthority("SCOPE_deleteIngredients")
   ...
 }
```

For each of the endpoints, the `.hasAuthority()` method is called to specify the required scope. Notice that the scopes are prefixed with `"SCOPE_"` to indicate that they should be matched against OAuth 2 scopes in the access token given on the request to those resources.

In that same configuration class, we'll also need to enable the resource server, as shown next:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
  http
    ...
     .and()
       .oauth2ResourceServer(oauth2 -> oauth2.jwt())
   ...
}
```

The `oauth2ResourceServer()` method here is given a lambda with which to configure the resource server. Here, it simply enables JWT tokens (as opposed to opaque tokens) so that the resource server can inspect the contents of the token to find what security claims it includes. Specifically, it will look to see that the token includes the `"writeIngredients"` and/or `"deleteIngredients"` scope for the two endpoints we've secured.

It won't trust the token at face value, though. To be confident that the token was created by a trusted authorization server on behalf of a user, it will verify the token's signature using the public key that matches the private key that was used to create the token's signature. We'll need to configure the resource server to know where to obtain the public key, though. The following property will specify the JWK set URL on the authorization server from which the resource server will fetch the public key:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: http://localhost:9000/oauth2/jwks
```

And now our resource server is ready! Build the Taco Cloud application and start it up. Then you can try it out using `curl` like this:

```
$ curl localhost:8080/ingredients \
       -H"Content-type: application/json" \
       -d'{"id":"CRKT", "name":"Legless Crickets", "type":"PROTEIN"}'
```

The request should fail with an HTTP 401 response code. That's because we've configured the endpoint to require the `"writeIngredients"` scope for that endpoint, and we've not pro-

vided a valid access token with that scope on the request.

To make a successful request and add a new ingredient item, you'll need to obtain an access token using the flow we used in the previous section, making sure that we request the `"writeIngredients"` and `"deleteIngredients"` scopes when directing the browser to the authorization server. Then, provide the access token in the `"Authorization"` header using `curl` like this (substituting `"$token"` for the actual access token):

```
$ curl localhost:8080/ingredients \
    -H"Content-type: application/json" \
    -d'{"id":"SHMP", "name":"Coconut Shrimp", "type":"PROTEIN"}' \
    -H"Authorization: Bearer $token"
```

This time the new ingredient should be created. You can verify that by using `curl` or your chosen HTTP client to perform a `GET` request to the /ingredients endpoint as follows:

```
$ curl localhost:8080/ingredients
[
  {
    "id": "FLTO",
    "name": "Flour Tortilla",
    "type": "WRAP"
  },

  ...

  {
    "id": "SHMP",
    "name": "Coconut Shrimp",
    "type": "PROTEIN"
  }
]
```

Coconut Shrimp is now included at the end of the list of all of the ingredients returned from the /ingredients endpoint. Success!

Recall that the access token expires after 5 minutes. If you let the token expire, requests will start returning HTTP 401 responses again. But you can get a new access token by making a request to the authorization server using the refresh token that you got along with the access token (substituting the actual refresh token for `"$refreshToken"`), as shown here:

```
$ curl localhost:9000/oauth2/token \
    -H"Content-type: application/x-www-form-urlencoded" \
```

```
    -d"grant_type=refresh_token&refresh_token=$refreshToken" \
    -u taco-admin-client:secret
```

With a newly created access token, you can keep on creating new ingredients to your heart's content.

Now that we've secured the /ingredients endpoint, it's probably a good idea to apply the same techniques to secure other potentially sensitive endpoints in our API. The /orders endpoint, for example, should probably not be open for any kind of request, even HTTP `GET` requests, because it would allow a hacker to easily grab customer information. I'll leave it up to you to secure the /orders endpoint and the rest of the API as you see fit.

Administering the Taco Cloud application using `curl` works great for tinkering and getting to know how OAuth 2 tokens play a part in allowing access to a resource. But ultimately we want a real client application that can be used to manage ingredients. Let's now turn our attention to creating an OAuth-enabled client that will obtain access tokens and make requests to the API.

8.4 Developing the client

In the OAuth 2 authorization dance, the client application's role is to obtain an access token and make requests to the resource server on behalf of the user. Because we're using OAuth 2's authorization code flow, that means that when the client application determines that the user has not yet been authenticated, it should redirect the user's browser to the authorization server to get consent from the user. Then, when the authorization server redirects control back to the client, the client must exchange the authorization code it receives for the access token.

First things first: the client will need Spring Security's OAuth 2 client support in its classpath. The following starter dependency makes that happen:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Not only does this give the application OAuth 2 client capabilities that we'll exploit in a moment, but it also transitively brings in Spring Security itself. This enables us to write some security configuration for the application. The following `SecurityFilterChain` bean sets up Spring Security so that all requests require authentication:

```
@Bean
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
  http
    .authorizeRequests(
       authorizeRequests -> authorizeRequests.anyRequest().authenticated()
    )
    .oauth2Login(
      oauth2Login ->
      oauth2Login.loginPage("/oauth2/authorization/taco-admin-client"))
    .oauth2Client(withDefaults());
  return http.build();
}
```

What's more, this `SecurityFilterChain` bean also enables the client-side bits of OAuth 2. Specifically, it sets up a login page at the path /oauth2/authorization/taco-admin-client. But this is no ordinary login page that takes a username and password. Instead, it accepts an authorization code, exchanges it for an access token, and uses the access token to determine the identity of the user. Put another way, this is the path that the authorization server will redirect to after the user has granted permission.

We also need to configure details about the authorization server and our application's OAuth 2 client details. That is done in configuration properties, such as in the following application.yml file, which configures a client named `taco-admin-client`:

```
spring:
  security:
    oauth2:
      client:
        registration:
          taco-admin-client:
            provider: tacocloud
            client-id: taco-admin-client
            client-secret: secret
            authorization-grant-type: authorization_code
            redirect-uri:
              ↪ "http://127.0.0.1:9090/login/oauth2/code/{registrationId}"
            scope: writeIngredients,deleteIngredients,openid
```

This registers a client with the Spring Security OAuth 2 client named `taco-admin-client`. The registration details include the client's credentials (`client-id` and `client-secret`), the grant type (`authorization-grant-type`), the scopes being requested (`scope`), and the redirect URI (`redirect-uri`). Notice that the value given to `redirect-uri` has a placeholder that references the client's registration ID, which is `taco-admin-client`. Consequently, the redirect URI is set

to **http://127.0.0.1:9090/login/oauth2/code/taco-admin-client**, which has the same path that we configured as the OAuth 2 login earlier.

But what about the authorization server itself? Where do we tell the client that it should redirect the user's browser? That's what the `provider` property does, albeit indirectly. The `provider` property is set to `tacocloud`, which is a reference to a separate set of configuration that describes the `tacocloud` provider's authorization server. That provider configuration is configured in the same application.yml file like this:

```
spring:
  security:
    oauth2:
      client:
...
        provider:
          tacocloud:
            issuer-uri: http://authserver:9000
```

The only property required for a provider configuration is `issuer-uri`. This property identifies the base URI for the authorization server. In this case, it refers to a server host whose name is `authserver`. Assuming that you are running these examples locally, this is just another alias for `localhost`. On most Unix-based operating systems, this can be added in your /etc/hosts file with the following line:

```
127.0.0.1 authserver
```

Refer to documentation for your operating system for details on how to create custom host entries if /etc/hosts isn't what works on your machine.

Building on the base URL, Spring Security's OAuth 2 client will assume reasonable defaults for the authorization URL, token URL, and other authorization server specifics. But, if for some reason the authorization server you're working with differs from those default values, you can explicitly configure authorization details like this:

```
spring:
  security:
    oauth2:
      client:
        provider:
          tacocloud:
            issuer-uri: http://authserver:9000
            authorization-uri: http://authserver:9000/oauth2/authorize
```

```
        token-uri: http://authserver:9000/oauth2/token
        jwk-set-uri: http://authserver:9000/oauth2/jwks
        user-info-uri: http://authserver:9000/userinfo
        user-name-attribute: sub
```

We've seen most of these URIs, such as the authorization, token, and JWK Set URIs already. The `user-info-uri` property is new, however. This URI is used by the client to obtain essential user information, most notably the user's username. A request to that URI should return a JSON response that includes the property specified in `user-name-attribute` to identify the user. Note, however, when using Spring Authorization Server, you do not need to create the endpoint for that URI; Spring Authorization Server will expose the user-info endpoint automatically.

Now all of the pieces are in place for the application to authenticate and obtain an access token from the authorization server. Without doing anything more, you could fire up the application, make a request to any URL on that application, and be redirected to the authorization server for authorization. When the authorization server redirects back, then the inner workings of Spring Security's OAuth 2 client library will exchange the code it receives in the redirect for an access token. Now, how can we use that token?

Let's suppose that we have a service bean that interacts with the Taco Cloud API using `RestTemplate`. The following `RestIngredientService` implementation shows such a class that offers two methods: one for fetching a list of ingredients and another for saving a new ingredient:

```
package tacos;

import java.util.Arrays;
import org.springframework.web.client.RestTemplate;
public class RestIngredientService implements IngredientService {

 private RestTemplate restTemplate;

 public RestIngredientService() {
  this.restTemplate = new RestTemplate();
 }

 @Override
 public Iterable<Ingredient> findAll() {
  return Arrays.asList(restTemplate.getForObject(
        "http://localhost:8080/api/ingredients",
        Ingredient[].class));
 }
 @Override
```

```
  public Ingredient addIngredient(Ingredient ingredient) {
    return restTemplate.postForObject(
        "http://localhost:8080/api/ingredients",
        ingredient,
        Ingredient.class);
  }

}
```

The HTTP `GET` request for the /ingredients endpoint isn't secured, so the `findAll()` method should work fine, as long as the Taco Cloud API is listening on localhost, port 8080. But the `addIngredient()` method is likely to fail with an HTTP 401 response because we've secured `POST` requests to /ingredients to require `"writeIngredients"` scope. The only way around that is to submit an access token with `"writeIngredients"` scope in the request's `Authorization` header.

Fortunately, Spring Security's OAuth 2 client should have the access token handy after completing the authorization code flow. All we need to do is make sure that the access token ends up in the request. To do that, let's change the constructor to attach a request interceptor to the `RestTemplate` it creates as follows:

```
public RestIngredientService(String accessToken) {
  this.restTemplate = new RestTemplate();
  if (accessToken != null) {
    this.restTemplate
        .getInterceptors()
        .add(getBearerTokenInterceptor(accessToken));
  }
}
private ClientHttpRequestInterceptor
        getBearerTokenInterceptor(String accessToken) {
  ClientHttpRequestInterceptor interceptor =
      new ClientHttpRequestInterceptor() {
    @Override
    public ClientHttpResponse intercept(
        HttpRequest request, byte[] bytes,
        ClientHttpRequestExecution execution) throws IOException {
      request.getHeaders().add("Authorization", "Bearer " + accessToken);
      return execution.execute(request, bytes);
    }
  };

  return interceptor;
}
```

The constructor now takes a `String` parameter that is the access token. Using this token, it attaches a client request interceptor that adds the `Authorization` header to every request made by the `RestTemplate` such that the header's value is `"Bearer"` followed by the token value. In the interest of keeping the constructor tidy, the client interceptor is created in a separate `private` helper method.

Only one question remains: where does the access token come from? The following bean method is where the magic happens:

```
@Bean
@RequestScope
public IngredientService ingredientService(
          OAuth2AuthorizedClientService clientService) {
 Authentication authentication =
       SecurityContextHolder.getContext().getAuthentication();

 String accessToken = null;

 if (authentication.getClass()
         .isAssignableFrom(OAuth2AuthenticationToken.class)) {
  OAuth2AuthenticationToken oauthToken =
        (OAuth2AuthenticationToken) authentication;
  String clientRegistrationId =
        oauthToken.getAuthorizedClientRegistrationId();
  if (clientRegistrationId.equals("taco-admin-client")) {
   OAuth2AuthorizedClient client =
      clientService.loadAuthorizedClient(
        clientRegistrationId, oauthToken.getName());
    accessToken = client.getAccessToken().getTokenValue();
  }
 }
 return new RestIngredientService(accessToken);
}
```

To start, notice that the bean is declared to be request-scoped using the `@RequestScope` annotation. This means that a new instance of the bean will be created on every request. The bean must be request-scoped because it needs to pull the authentication from the `SecurityContext`, which is populated on every request by one of Spring Security's filters; there is no `SecurityContext` at application startup time when default-scoped beans are created.

Before returning a `RestIngredientService` instance, the bean method checks to see that the authentication is, in fact, implemented as `OAuth2AuthenticationToken`. If so, then that means it will have the token. It then verifies that the authentication token is for the client

named `taco-admin-client`. If so, then it extracts the token from the authorized client and passes it through the constructor for `RestIngredientService`. With that token in hand, `RestIngredientService` will have no trouble making requests to the Taco Cloud API's endpoints on behalf of the user who authorized the application.

Summary

- OAuth 2 security is a common way to secure APIs that is more robust than simple HTTP Basic authentication.
- An authorization server issues access tokens for a client to act on behalf of a user when making requests to an API (or on its own behalf in the case of client token flow).
- A resource server sits in front of an API to verify that valid, nonexpired tokens are presented with the scope necessary to access API resources.
- Spring Authorization Server is an experimental project that implements an OAuth 2 authorization server.
- Spring Security provides support for creating a resource server, as well as creating clients that obtain access tokens from the authorization server and pass those tokens when making requests through the resource server.

---

[1] Depending on the database and schema in play, integrity constraints may prevent a deletion from happening if an ingredient is already part of an existing taco. But it still may be possible to delete an ingredient if the database schema allows it.

[2] See **http://mng.bz/QqGR**.

[3] See **https://datatracker.ietf.org/doc/html/rfc7517**.

[4] Notice that this and all URLs in this chapter are using "http://" URLs. This makes local development and testing easy. But in a production setting, you should always use "https://" URLs for increased security.

Previous chapter

< 7 Creating REST services

Next chapter

9 Sending messages asynchronously >